Indian Institute of Science, Bengaluru

Department of Electrical Engineering

COURSE ASSIGNMENT

AI-Assisted Load Flow Simulator Development

Design, Implementation and Performance Evaluation

Submitted by: **Siddharth A S**
Mtech Electrical Engineering
SR No. 27992
siddharthas2@iisc.ac.in

Course: E4 234 - Advanced Power System Analysis

Submitted to: Dr. Sarasij Das

8th November 2026

1. Introduction and objectives

This work implements a desktop AC load-flow simulator using the Newton–Raphson (NR) method in polar coordinates. The software combines an editable single-line diagram with an independent numerical engine for bus voltages, power injections, branch flows, and system losses. Python, NumPy, and SciPy are used for computation, while PySide6 is employed for the graphical user interface. Reference power-flow packages are used for numerical verification.

The simulator was developed with the assistance of OpenAI's ChatGPT, using both the Go and Plus subscription plans. AI assistance was utilized for code generation, debugging, software development, and documentation. The implemented algorithms and numerical results were subsequently tested and validated against standard IEEE power system test cases.

The objectives are to

- Form a network admittance model with lines, transformer taps and shunts
- Solve the nonlinear active/reactive power-balance equations with an analytical Jacobian
- Provide accessible network editing and results
- verify results and computational performance on the IEEE 14-, 30-, 57- and 118-bus test systems

2. Numerical formulation and implementation

2.1 Network model and per-unit representation

Bus and branch parameters are stored on a common MVA base. Transmission lines use a series impedance and total shunt charging susceptance (π equivalent). Transformer models include off-nominal magnitude and phase-shift taps, while fixed bus shunts contribute directly to the admittance matrix. The bus admittance matrix is assembled in sparse form. In-service status and bus type (slack, PV, PQ) determine which quantities are specified and solved.

2.2 Newton–Raphson power-flow equations

For bus voltage V_i and network admittance matrix Y, the complex injected power is

$$S_i = V_i \overline{(YV)_i} = P_i + jQ_i. \tag{1}$$

The mismatch vector contains specified minus calculated active-power injections at non-slack buses, followed by reactive-power mismatches at PQ buses. The polar-coordinate Jacobian is assembled from four analytical derivative blocks:

$$\underbrace{\begin{bmatrix}\Delta P\\ \Delta Q\end{bmatrix}}_{\Delta S} = \underbrace{\begin{bmatrix}\dfrac{\partial P}{\partial \theta} & \dfrac{\partial P}{\partial |V|}\\ \dfrac{\partial Q}{\partial \theta} & \dfrac{\partial Q}{\partial |V|}\end{bmatrix}}_{J} \underbrace{\begin{bmatrix}\Delta\theta\\ \Delta|V|\end{bmatrix}}_{\Delta x}. \tag{2}$$

Each NR correction is obtained by sparse LU factorization.

After solving $J\Delta x = \Delta S$, voltage angles and the PQ-bus voltage magnitudes are updated until the largest mismatch is below the selected tolerance. Optional voltage-dependent ZIP/constant-impedance loads and generator reactive-limit switching are incorporated in the operating-point calculation.

2.3 Software architecture



The numerical modules are separated into network data, Y_{bus} assembly, power and mismatch calculation, Jacobian construction, linear solution, voltage correction, limit handling, and final flow/output calculations. The GUI collects and validates data, invokes the solver, then presents voltages, flows, violations and convergence results.

3. Verification against reference power flow

The same MATPOWER-formatted input cases were used for the simulator and PYPOWER reference solves. Comparisons were made for voltage magnitudes, relative voltage angles, generator P/Q, branch P/Q and total losses. All six categories passed for all four systems. Reported errors below are maximum absolute differences, except that voltage angles are referenced to each solution's slack-bus angle.

Network	Max $\lvert\Delta V\rvert$ (pu)	Max relative $\lvert\Delta\theta\rvert$ (°)	Max branch-flow error (MW/MVAr)	**Result**
IEEE 14	6.661×10^{-16}	2.842×10^{-14}	3.606×10^{-13}	**PASS**
IEEE 30	1.332×10^{-15}	9.948×10^{-14}	1.389×10^{-12}	**PASS**
IEEE 57	2.038×10^{-12}	1.122×10^{-10}	1.779×10^{-10}	**PASS**
IEEE 118	2.109×10^{-14}	1.802×10^{-13}	4.630×10^{-11}	**PASS**

The largest reported generator-output error was 9.902×10^{-11} MW/MVAr (IEEE 57); the largest active-loss error was 3.721×10^{-11} MW (IEEE 57). These discrepancies are far below the assignment acceptance thresholds. The most recent uploaded project also passed 58 automated tests, covering the solver, import/export, network utilities and other component logic.

4. Runtime performance study

The four IEEE cases were benchmarked on the my Windows machine using Python 3.14. The following are the measured wall-clock totals and reported time per NR update; timings vary with hardware, library versions and initialization.

System	NR updates	Total time (ms)	Time/update (ms)	Final max mismatch (pu)
IEEE 14	2	3.673	1.837	1.316×10^{-10}
IEEE 30	2	7.382	3.691	3.469×10^{-9}
IEEE 57	3	33.148	11.049	3.449×10^{-12}
IEEE 118	3	90.636	30.212	1.506×10^{-12}

All four benchmark runs converged below the specified 10^{-8} p.u. mismatch tolerance. The IEEE 118-bus run used three updates, satisfying the assignment limit of no more than seven updates (flat start, Q limits off). The validation script separately reported four iterations per case; its iteration counts should not be conflated with the benchmark update counts without accounting for the scripts' settings and counting conventions.

5. GUI capabilities and engineering checks

The desktop interface supports interactive single-line-diagram creation, editing of buses and connected equipment, MATPOWER and IEEE CDF imports, and multiple study/result views. JSON project saving, undo/redo, element deletion, voltage and branch-loading indicators, result tables, convergence plotting, and CSV/Excel/PDF/HTML exports are implemented in the reviewed source. The user separately confirmed that zoom and deletion operations work after their fixes.

Checks used to reduce engineering errors. The network validation and input editors address invalid impedances, disconnections, bus-type/source consistency and other malformed data. Unit utilities convert physical line or transformer inputs to the system per-unit base. Generator reactive limit handling supports PV/PQ transitions; voltage-dependent loads contribute to the mismatch and its derivatives. Output quantities are kept distinct from generator schedules and bus net injections.

6. Observations, limitations and interpretation

Numerical accuracy. The test comparisons demonstrate very close agreement with a reference implementation across four increasingly large networks, including transformer taps and shunts. The growing execution time with system size is expected because the size and cost of building/factorizing the Jacobian increase; these measurements alone do not establish asymptotic complexity.

Scope. The successful tests validate the tested models and settings. Limitations identified during review include per-bus aggregation of multiple generator outputs, some incomplete advanced property fields and right-click/snap-to-grid interactions.

7. Conclusion

The project demonstrates an independently implemented polar Newton–Raphson AC load-flow engine, integrated into a graphical network editor. Numerical comparisons for IEEE 14, 30, 57 and 118 systems meet the stated tolerances for bus voltages, relative angles, generator and branch outputs and losses. The IEEE 118 case also satisfies the specified iteration limit. A modular architecture and automated test suite support incremental verification.